Contents

1 Introduction **5**
1.1 What is the ESP32 RC Controller System . . . 5
1.2 Who This Guide is For . . . 5
1.3 What You Can Build . . . 5

2 Required Components **7**
2.1 Hardware Requirements . . . 7
2.2 Optional Components . . . 7
2.3 Software Requirements . . . 7

3 Hardware Setup **8**
3.1 ESP32 Board . . . 8
3.2 Motor Driver Connection . . . 8
3.3 Servo Connections . . . 8
3.4 Power Supply . . . 9

4 Uploading the Firmware **10**
4.1 Install the Arduino IDE . . . 10
4.2 Install ESP32 Board Support . . . 10
4.3 Download the Firmware . . . 11
4.4 Open the Firmware Project . . . 11
4.5 Select the ESP32 Board . . . 11
4.6 Configure Board Parameters . . . 11
4.7 Connect the ESP32 Board . . . 12
4.8 Compile the Firmware . . . 12
4.9 Upload the Firmware . . . 12
4.10 Verify Firmware Operation . . . 13
4.11 Next Step . . . 13

5 Connecting the Android Application **14**
5.1 Install the Application . . . 14
5.2 Enable Bluetooth . . . 14
5.3 Connect to the ESP32 . . . 14
5.4 Connection Status . . . 14
5.5 Debug Mode (Viewing Bluetooth Communication) . . . 14
5.5.1 Enable Debug Mode . . . 14
5.5.2 Open the Serial Monitor . . . 15
5.5.3 Debug Output . . . 15
5.5.4 Troubleshooting Using Debug Mode . . . 15
5.6 Using Debug Mode to Verify Communication . . . 16

5.6.1 Step 1: Enable Debug Mode . . . 16
5.6.2 Step 2: Open the Serial Monitor . . . 16
5.6.3 Step 3: Move Controls in the App . . . 16
5.7 Debug Output Types . . . 17
5.7.1 Joystick Debug (Sticks) . . . 17
5.7.2 Knob Debug . . . 17
5.7.3 Switch Debug . . . 18
5.7.4 Event Debug . . . 18
5.8 Debugging the Telemetry System . . . 19
5.8.1 Normal Operation . . . 19
5.8.2 Numeric Panel Debug Mode . . . 19
5.8.3 Indicator Debug Mode . . . 21
5.8.4 Plot Debug Mode . . . 21
5.8.5 How the Telemetry Debug System Works . . . 22
5.9 Why Debug Mode is Important . . . 22
5.10 Customizing Telemetry Labels in the Android Application . . . 22
5.10.1 Where the Labels Are Defined . . . 22
5.10.2 Plot Labels . . . 23
5.10.3 Numeric Panel Labels . . . 23
5.10.4 Indicator Labels . . . 23
5.10.5 When the Labels Are Sent . . . 24
5.10.6 Important Notes . . . 24

6 Basic Controls **25**
6.1 RC Screen Layout . . . 25
6.2 Joystick Control . . . 25
6.3 Switches (Discrete Channels) . . . 26
6.4 Knobs (Analog Channels) . . . 26
6.5 Center Buttons . . . 26
6.6 Indicators and Telemetry . . . 26
6.7 Summary . . . 27

7 Customizing Your Project **28**
7.1 Adding Sensors . . . 28
7.2 Using I2C Devices . . . 28
7.3 Expanding Outputs . . . 28

8 Example Projects **29**
8.1 RC Car (Differential Drive) . . . 29
8.2 Camera Control Platform (Pan/Tilt Head) . . . 30
8.3 Sensor Rover (Telemetry-First Build) . . . 31
8.4 Custom Robotics Platform (Modular, Expandable) . . . 31
8.5 Project Planning Checklist (Recommended) . . . 32

9 Wiring Diagrams **33**
9.1 Basic Robot Wiring . . . 33
9.2 Servo Control Example . . . 33
9.3 LED and Buzzer Outputs . . . 34
9.4 Sensor Connections . . . 34

10 Troubleshooting **35**

10.1 Before You Start: Quick Checklist . . . 35
10.2 ESP32 Firmware Does Not Upload . . . 35
10.3 ESP32 Powers On but Resets Randomly . . . 36
10.4 Bluetooth Pairing and Connection Problems (Android ↔ ESP32) . . . 36
10.5 Controls Feel Wrong in the App (Android Side) . . . 37
10.6 Motors Do Not Move . . . 37
10.7 Servo Does Not Respond or Jitters . . . 38
10.8 Telemetry Data Not Appearing in the App . . . 38
10.9 I2C Devices Not Detected (MCP23017 or Sensors) . . . 38
10.10 General Debugging Tips (Recommended Workflow) . . . 38

11 Advanced Features **40**
11.1 Protocol Overview: Commands, Events, and Telemetry . . . 40
11.2 Telemetry System (Panels, Indicators, Plots) . . . 40
11.2.1 Telemetry categories . . . 40
11.2.2 Telemetry update rates (practical meaning) . . . 41
11.2.3 Automatic configuration handshake (labels) . . . 41
11.2.4 Panel resend window (robustness feature) . . . 41
11.3 Telemetry Sources and Debug Injection . . . 41
11.4 Project Modes: ESP32-Only vs MCP23017 Expansion . . . 42
11.5 Using the MCP23017 GPIO Expander (I2C) . . . 42
11.5.1 Key details (from the implementation) . . . 42
11.5.2 Practical applications . . . 42
11.6 I2C Sensor Support (Examples Included) . . . 42
11.7 Analog Input Design for Bluetooth Compatibility . . . 42
11.8 Feature Flags (Build-Time Modular Firmware) . . . 43
11.9 Putting It All Together: Example “Advanced Build” . . . 43

12 Practical Control and Telemetry Examples **44**
12.1 Introduction . . . 44
12.2 Control Data from the Android Application . . . 44
12.2.1 RC state packet (`rcStatePacket`) . . . 44
12.2.2 Event packet (`rcEventPacket`) . . . 44
12.3 How the ESP32 Receives Packets . . . 45
12.4 Where Control Values are Applied . . . 45
12.5 Example 1: Steering Control Using the Left Joystick . . . 45
12.6 Example 2: Differential Drive Motor Speed Control . . . 46
12.6.1 Direction + speed mapping (implemented behavior) . . . 46
12.6.2 Smooth ramping (advanced behavior) . . . 46
12.7 Example 3: Camera Pan Control . . . 46
12.8 Example 4: Using Knobs to Control LED and Buzzer PWM . . . 47
12.9 Example 5: Controlling Digital Outputs with Switches . . . 47
12.10 Example 6: Handling Button Events (Pulse Outputs) . . . 48
12.10.1 Non-blocking pulses . . . 48
12.11 Telemetry Data Sent to the Android Application . . . 48
12.11.1 Telemetry types and rates (implemented) . . . 48
12.11.2 Config-on-connect . . . 48
12.12 Example 7: Battery Telemetry as an Indicator . . . 49
12.13 Example 8: Telemetry Panels from Analog Inputs . . . 49
12.14 Example 9: Plot Signals (0..255 Fast Graphs) . . . 49

12.15 Example 10: Digital Mask Indicators . . . 50
12.16 Summary . . . 50

13 Control Layer and Hardware Abstraction **51**
13.1 Purpose of This Chapter . . . 51
13.2 Layered Architecture (High-Level View) . . . 51
13.3 Control Packets and State Structures . . . 52
13.4 Control Layer (`control.cpp`) . . . 52
13.4.1 Responsibilities . . . 52
13.4.2 Main loop: `controlUpdate()` . . . 53
13.4.3 Switch bit mask decoding . . . 53
13.4.4 Event handling: `controlHandleEvent(eventId)` . . . 53
13.5 Hardware Abstraction Layer (`hal_output.cpp`) . . . 53
13.5.1 Responsibilities . . . 53
13.5.2 PWM outputs (examples) . . . 54
13.5.3 Motor control: direction + dead-band + ramping . . . 54
13.5.4 Switch outputs: direct GPIO vs MCP23017 . . . 54
13.5.5 Pulse outputs (momentary actions) . . . 54
13.6 Pin Mapping (`pins.h`) and Portability . . . 54
13.7 Summary . . . 55

14 Wiring Diagram Explained (Repository Reference) **56**
14.1 Quick Summary . . . 56
14.2 Diagram (Included) . . . 57
14.3 Straight Explanation of Each Group (What it does) . . . 58
14.3.1 I2C (SDA 21, SCL 22) . . . 58
14.3.2 Analog telemetry inputs (NUM1/NUM2/AIN1/AIN2) . . . 58
14.3.3 Digital indicator inputs (IND1–IND4) . . . 58
14.3.4 PWM outputs (PWM LX/LY/RX/RY/KL/KR) . . . 58
14.3.5 Switch outputs (SW1–SW6) . . . 58
14.3.6 Pulse/event outputs . . . 58
14.4 Resume (One Paragraph) . . . 58

Chapter 1

Introduction

1.1 What is the ESP32 RC Controller System

The ESP32 RC Controller system allows you to control electronics projects using an Android phone and an ESP32 microcontroller.

The Android application communicates with the ESP32 using Bluetooth. Control commands from the phone are sent to the ESP32, which then updates motors, servos, LEDs, and other devices connected to the board.

At the same time, the ESP32 can send information back to the phone, such as:

- Sensor values
- Button states
- System status
- Real-time telemetry

This creates a bidirectional communication system between the Android application and the ESP32.

1.2 Who This Guide is For

This guide is intended for:

- Students learning electronics
- Robotics hobbyists
- Makers building DIY projects
- Beginners using the ESP32 platform

Only basic knowledge of the Arduino IDE and simple electronic wiring is required.

1.3 What You Can Build

Using the RC Controller system you can build many types of projects:

- RC cars

- Robots
- Camera pan systems
- Sensor monitoring platforms
- Bluetooth-controlled devices

The firmware was designed to be modular, allowing users to extend the system with additional sensors, motors, and control devices.

Chapter 2

Required Components

2.1 Hardware Requirements

To use the RC Controller system you will need the following hardware:

- ESP32 development board
- Android smartphone
- Motor driver module (L298N, BTS7960, or similar)
- DC motors or servos
- Power supply or battery
- Jumper wires

2.2 Optional Components

The system can also support additional devices:

- LEDs
- Buzzer
- I2C sensors
- MCP23017 GPIO expansion module
- Analog sensors

2.3 Software Requirements

You will also need the following software tools:

- Arduino IDE
- ESP32 board support package
- RC Controller Android application

Chapter 3

Hardware Setup

3.1 ESP32 Board

The ESP32 acts as the main controller for the system. It receives commands from the Android application via Bluetooth and controls the connected hardware.

Typical ESP32 development boards include:

- ESP32 DevKit
- ESP32-WROOM boards
- ESP32 NodeMCU boards

3.2 Motor Driver Connection

Motors cannot be powered directly from the ESP32 pins. A motor driver is required.

The motor driver receives control signals from the ESP32 and supplies power to the motors.

Typical connections include:

- Motor direction pins
- PWM speed control pins
- Power input
- Motor outputs

3.3 Servo Connections

Servo motors can be connected directly to the ESP32 PWM pins.

Servos can be used for:

- Steering
- Camera pan control
- Robotic arms

3.4 Power Supply

Motors and the ESP32 should be powered from an appropriate battery or power supply.

Always ensure that all devices share a common ground connection.

Chapter 4

Uploading the Firmware

This chapter explains how to install the development environment and upload the firmware to the ESP32 microcontroller. The firmware enables Bluetooth communication with the mobile application and provides the control logic for motors, servos, sensors, and telemetry.

The procedure uses the Arduino IDE, which provides a simple environment for compiling and uploading firmware to ESP32 development boards.

4.1 Install the Arduino IDE

Download and install the Arduino IDE from the official website:

`https://www.arduino.cc/en/software`

Both Arduino IDE 1.8.x and Arduino IDE 2.x are supported, but Arduino IDE 2.x is recommended because it provides improved performance and a modern user interface.

After installing the software, launch the Arduino IDE to continue with the ESP32 configuration.

4.2 Install ESP32 Board Support

The Arduino IDE requires the ESP32 board package in order to compile firmware for ESP32 microcontrollers.

Open the Arduino IDE and navigate to:

```
File -> Preferences
```

Locate the field labeled **Additional Boards Manager URLs** and add the following address:

```
https://dl.espressif.com/dl/package_esp32_index.json
```

Press **OK** to save the configuration.

Next install the ESP32 board definitions:

```
Tools -> Board -> Boards Manager
```

Search for:

```
ESP32 by Espressif Systems
```

Install the board package.

For best compatibility with this project the following version is recommended:

```
ESP32 Core Version 2.0.x
```

This version provides stable Bluetooth support and works well with the firmware architecture used in this project.

4.3 Download the Firmware

Download the project source code from the GitHub repository:

`https://github.com/MICSBOL/ESP32_BT_Controller`

You can download the project by clicking the **Code** button and selecting **Download ZIP**, or by cloning the repository using Git.

After downloading the project, extract the files to a folder on your computer.

4.4 Open the Firmware Project

Open the Arduino IDE and select:

```
File -> Open
```

Navigate to the folder containing the firmware and open the main project file:

```
ESP32_BT_Controller.ino
```

When the file is opened, the Arduino IDE will automatically load all additional source files used by the firmware, including the modular components responsible for control logic, communication, telemetry, and hardware interfaces.

4.5 Select the ESP32 Board

Configure the board type used for compilation.

Navigate to:

```
Tools -> Board
```

Select:

```
ESP32 Dev Module
```

This configuration is compatible with most ESP32 development boards available on the market.

4.6 Configure Board Parameters

For reliable operation it is recommended to use the following board configuration:

- Board: ESP32 Dev Module
- Upload Speed: 921600

- CPU Frequency: 240 MHz
- Flash Frequency: 80 MHz
- Flash Mode: QIO
- Flash Size: 4MB
- Partition Scheme: Default 4MB
- Core Debug Level: None
- PSRAM: Disabled

These settings ensure stable compilation and firmware upload for most ESP32 boards.

4.7 Connect the ESP32 Board

Connect the ESP32 development board to your computer using a USB cable.

After connecting the board, select the serial port used by the device:

```
Tools -> Port
```

Typical port names include:

```
COM3
COM4
/dev/ttyUSB0
/dev/ttyACM0
```

Select the port corresponding to your ESP32 device.

4.8 Compile the Firmware

Before uploading the firmware it is recommended to verify that the code compiles correctly.

Click the **Verify** button in the Arduino IDE.

The IDE will compile all source files used by the project. If the compilation completes successfully the console will display a message indicating the memory usage of the program.

4.9 Upload the Firmware

Click the **Upload** button to transfer the compiled firmware to the ESP32.

During the upload process the IDE will compile the firmware and send it to the microcontroller through the USB connection.

Some ESP32 boards require pressing the **BOOT** button when the upload begins. If the upload fails, hold the BOOT button while the firmware is being uploaded.

After the upload finishes the IDE will display a message indicating that the ESP32 is restarting.

4.10 Verify Firmware Operation

Open the Serial Monitor to confirm that the firmware is running.

Navigate to:

```
Tools -> Serial Monitor
```

Set the serial communication speed to:

```
115200
```

When the ESP32 starts you should see status messages printed to the console indicating that the Bluetooth controller has started and is ready to accept connections from the mobile application.

4.11 Next Step

After the firmware has been uploaded successfully, the next step is to connect the mobile application to the ESP32 and begin controlling the device.

The connection process is described in Chapter 5.

Chapter 5

Connecting the Android Application

5.1 Install the Application

Install the RC Controller Android application on your smartphone.

5.2 Enable Bluetooth

Make sure Bluetooth is enabled on your phone.

5.3 Connect to the ESP32

Open the RC Controller App and select the ESP32 device from the list of available Bluetooth devices.

Once connected, the control interface will appear on the screen.

5.4 Connection Status

The application will display the connection status.

If the connection is successful, control commands can be sent to the ESP32.

5.5 Debug Mode (Viewing Bluetooth Communication)

The firmware includes an optional debug system that allows users to monitor the communication between the Android application and the ESP32.

This feature is useful when troubleshooting connection problems or verifying that commands from the application are being received correctly.

5.5.1 Enable Debug Mode

Open the file:

```
debug_config.h
```

Inside this file you will find a configuration flag for enabling debug output.

Enable debugging by setting the flag to:

```
#define DEBUG_ENABLED 1
```

If debugging is disabled, the value will typically be:

```
#define DEBUG_ENABLED 0
```

After changing this value, upload the firmware again to the ESP32.

5.5.2 Open the Serial Monitor

Open the Arduino Serial Monitor:

```
Tools -> Serial Monitor
```

Set the baud rate to:

```
115200
```

Restart the ESP32 to view the startup messages.

5.5.3 Debug Output

When debug mode is enabled the ESP32 will print diagnostic messages to the Serial Monitor.

Typical messages may include:

```
[DEBUG] Bluetooth initialized
[DEBUG] Waiting for client connection
[DEBUG] Client connected
```

When commands are received from the Android application, the ESP32 may display messages similar to:

```
[DEBUG] Received command: BTN_A
[DEBUG] Joystick X: 120
[DEBUG] Joystick Y: 85
```

These messages confirm that the Bluetooth communication between the smartphone and the ESP32 is working correctly.

5.5.4 Troubleshooting Using Debug Mode

If the Android application connects but the robot does not respond:

- Verify that debug messages appear when pressing buttons in the application.
- If no messages appear, the ESP32 may not be receiving Bluetooth data.
- Check that the correct ESP32 device is selected in the application.
- Verify that the firmware was uploaded successfully.

If debug messages appear but the hardware does not respond, the issue is likely related to the wiring or pin configuration.

5.6 Using Debug Mode to Verify Communication

The firmware includes a built-in debugging system that allows the user to see the data being received from the Android application.

This is extremely useful when verifying that the Bluetooth connection is working correctly and that the ESP32 is receiving control commands.

All debug messages are displayed in the Arduino IDE Serial Monitor.

5.6.1 Step 1: Enable Debug Mode

Open the file:

```
debug_config.h
```

Locate the following line:

```
#define DEBUG_ENABLED 1
```

Set the value to:

- `1` to enable debug output
- `0` to disable debug output

When debugging is enabled, the firmware will print control data received from the Android application.

Upload the firmware again after making any changes.

5.6.2 Step 2: Open the Serial Monitor

In the Arduino IDE open:

```
Tools -> Serial Monitor
```

Set the baud rate to:

```
115200
```

Restart the ESP32 or press the reset button.

The Serial Monitor will begin displaying debug messages.

5.6.3 Step 3: Move Controls in the App

After connecting the Android application to the ESP32, move the controls on the screen:

- Move the joysticks
- Rotate knobs
- Toggle switches
- Press buttons

The ESP32 will print the received data in the Serial Monitor.

5.7 Debug Output Types

The debug system can print different categories of information depending on the configuration in `debug_config.h`.

5.7.1 Joystick Debug (Sticks)

If the following option is enabled:

```
#define DBG_STICKS 1
```

The ESP32 will print the joystick positions when they change.

Example output:

```
L Stick X: 512    L Stick Y: 510
L Stick X: 650    L Stick Y: 480

R Stick X: 300    R Stick Y: 700
```

These values represent the position of the left and right joystick on the Android application.

Typical range:

- Minimum: around 0
- Center: around 512
- Maximum: around 1023

If the numbers change when you move the joystick, the communication is working correctly.

5.7.2 Knob Debug

If the following option is enabled:

```
#define DBG_KNOBS 1
```

The ESP32 will print knob values.

Example output:

```
Left Knob: 340
Left Knob: 512
Left Knob: 890

Right Knob: 120
Right Knob: 780
```

These values correspond to the knob rotation in the Android application.

Knobs can be used to control parameters such as:

- Motor speed
- Servo angle
- Light intensity
- Camera tilt

5.7.3 Switch Debug

If the following option is enabled:

```
#define DBG_SWITCHES 1
```

The ESP32 will display the switch status.

Example output:

```
Switch Byte: 0x05
S1 = ON
S2 = OFF
S3 = ON
S4 = OFF
S5 = OFF
S6 = OFF
```

Each switch corresponds to a toggle button in the Android application.

These switches are commonly used for:

- Turning lights on and off
- Activating relays
- Changing driving modes
- Enabling autonomous functions

5.7.4 Event Debug

If the following option is enabled:

```
#define DBG_EVENTS 1
```

The ESP32 will print a message when an event packet arrives.

Example output:

```
--- EVENT PACKET (PRESS) ---
Event ID: 0x03
Checksum: 0x7A
--------------
```

Events are triggered when the user presses a button in the Android application.

Typical uses include:

- Activating a special function
- Resetting a system
- Triggering a sound or animation
- Launching an action on the robot

5.8 Debugging the Telemetry System

The firmware includes a telemetry system that allows the ESP32 to send real-time information back to the Android application.

Telemetry can include:

- Numeric values
- Indicators
- Real-time plots

For development and testing, the firmware provides a telemetry debug system that allows the user to simulate telemetry values using the Arduino Serial Monitor.

This system is configured in the file:

`debug_config.h`

Inside this file you can select the telemetry debug mode.

```
#define DBG_NONE        0
#define DBG_PANEL       1
#define DBG_INDICATOR   2
#define DBG_PLOT        3

#define TELEMETRY_DEBUG_MODE DBG_INDICATOR
```

Only one debug mode should be active at a time.

5.8.1 Normal Operation

```
#define TELEMETRY_DEBUG_MODE DBG_NONE
```

In this mode the ESP32 reads telemetry values from real hardware sensors.

Examples include:

- analog sensors
- battery voltage measurement
- digital inputs

This mode should be used when the robot is fully built.

5.8.2 Numeric Panel Debug Mode

```
#define TELEMETRY_DEBUG_MODE DBG_PANEL
```

This mode allows testing the numeric telemetry panels without connecting any sensors.

When this mode is enabled, values entered in the Arduino Serial Monitor are parsed and sent to the Android application as numeric panel telemetry.

Serial Monitor Input Format

Open the Arduino Serial Monitor and send values using the following format:

```
LEFT_VALUE,RIGHT_VALUE
```

Both values may include a decimal part.

Example inputs:

```
12.3,45.8
120.0,350.5
5,10
```

Value Conversion

The values entered in the Serial Monitor are internally converted to the telemetry format used by the firmware.

Each value is multiplied by 10 and converted to an integer before being transmitted to the Android application.

Examples:

- 12.3 becomes 123
- 45.8 becomes 458
- 120.0 becomes 1200

On the Android side, the received integer value is divided by 10 to display the correct decimal value on the numeric panel.

Valid Range

The allowed range for each numeric value is:

- 0.0 to 6553.5

This range corresponds to the maximum value that can be transmitted using a 16-bit unsigned integer with one decimal place of precision.

Example

If the following value is entered in the Serial Monitor:

```
123.4,56.7
```

The firmware will transmit:

```
1234,567
```

The Android application will then display:

```
123.4   56.7
```

on the left and right numeric panels respectively.

This debug mode is useful for verifying the correct operation of the telemetry transmission and the Android user interface before connecting real sensors.

5.8.3 Indicator Debug Mode

```
#define TELEMETRY_DEBUG_MODE DBG_INDICATOR
```

This mode allows testing indicator widgets.

Send values through the Serial Monitor using this format:

```
ANALOG_VALUE,BATTERY_LEVEL,DIGITAL_MASK
```

Example:

```
45,80,5
```

Value ranges:

- Analog value: 0 to 100
- Battery level: 0 to 100
- Digital mask: 0 to 255

The digital mask represents the state of digital indicators using bit flags.

Example values:

```
1
2
4
8
```

Each value activates a different indicator.

Multiple indicators can be activated by combining values.

Example:

```
3
```

Binary representation:

```
00000011
```

This activates the first two indicators.

5.8.4 Plot Debug Mode

```
#define TELEMETRY_DEBUG_MODE DBG_PLOT
```

This mode simulates three telemetry signals that are displayed as real-time graphs in the Android application.

Send values using the Serial Monitor:

```
VALUE1,VALUE2,VALUE3
```

Example:

```
10,40,80
```

Valid range:

- 0 to 255

These values will appear in the telemetry plots of the application.

5.8.5 How the Telemetry Debug System Works

The debug values are read by the module:

`telemetry_source.cpp`

This module replaces real sensor values with values received through the Serial interface when a debug mode is active.

Telemetry packets are created and transmitted by:

`telemetry.cpp`

This module builds telemetry packets and sends them to the Android application using Bluetooth.

This architecture allows the telemetry interface to be tested without connecting any physical sensors.

5.9 Why Debug Mode is Important

Debug mode allows users to quickly verify that:

- The Android application is connected
- Bluetooth data is being received
- Control values are changing correctly

If values appear in the Serial Monitor but the hardware does not respond, the issue is likely related to the hardware wiring or firmware configuration rather than the Bluetooth connection.

5.10 Customizing Telemetry Labels in the Android Application

The Android application automatically displays labels for telemetry values such as numeric panels, indicators, and plots.

These labels are defined in the ESP32 firmware and sent to the application when the Bluetooth connection is established.

This allows each project to customize the interface without modifying the Android application.

5.10.1 Where the Labels Are Defined

Telemetry labels are defined inside the file:

`telemetry.cpp`

Inside the function responsible for sending the telemetry configuration, you will find the following variables:

```
const char* plotNames[] = { "Volts", "Amps", "RPMs" };
const char* panelNames[] = { "Left", "Right" };
const char* indicatorNames[] = { "Throttle", "Battery" };
```

These arrays define the labels that appear in the Android application's RC screen.

5.10.2 Plot Labels

The `plotNames` array defines the names of the real-time graphs.

Example:

```
const char* plotNames[] = { "Volts", "Amps", "RPMs" };
```

In the Android application this will create three plots labeled:

- Volts
- Amps
- RPMs

You can rename them to match your sensors.

Example for a robot car:

```
const char* plotNames[] = { "Speed", "Motor Temp", "Battery Current" };
```

5.10.3 Numeric Panel Labels

The `panelNames` array defines the labels for the large numeric displays.

Example:

```
const char* panelNames[] = { "Left", "Right" };
```

Typical uses include:

- Motor speed
- Distance
- RPM
- Temperature

Example modification:

```
const char* panelNames[] = { "Motor RPM", "Battery Voltage" };
```

Note: It's recommended to use a maximum of 12 characters.

5.10.4 Indicator Labels

The `indicatorNames` array defines the labels for the indicator widgets.

Example:

```
const char* indicatorNames[] = { "Throttle", "Battery" };
```

Possible uses include:

- Throttle position
- Battery level
- Temperature

- Signal strength

Example modification:

```
const char* indicatorNames[] = { "Engine Load", "Battery Level" };
```

5.10.5 When the Labels Are Sent

The telemetry labels are transmitted only once after the Android application connects to the ESP32.

The firmware sends a telemetry configuration packet that includes:

- plot labels
- numeric panel labels
- indicator labels

After receiving this packet, the Android application automatically updates the RC screen interface.

5.10.6 Important Notes

- The number of labels must match the number of telemetry variables.
- After modifying the labels you must upload the firmware again.
- The Android application will update the labels automatically after reconnecting.

Chapter 6

Basic Controls

This chapter describes the main operator controls available on the *RC Screen*. The interface is organized into left and right controller sides plus a central telemetry area, allowing you to drive the robot while monitoring system status in real time.

6.1 RC Screen Layout

The RC Screen is composed of three main regions:

- **Left Controller Side:** primary controls (joystick mode dependent), switches, knob, and a top status widget.
- **Center Display:** a telemetry plot used to visualize live data.
- **Right Controller Side:** secondary controls (joystick mode dependent), switches, knob, and a top status widget.

A background panel provides contrast for readability, and the screen is intended to be used in **landscape orientation** for ergonomic spacing.

6.2 Joystick Control

Each controller side includes a virtual joystick. The joystick behavior depends on the selected **stick mode** (configured in user settings):

- **Spring mode:** the joystick returns to center when released (useful for momentary input such as steering or speed trim).
- **Hold mode:** the joystick keeps its position when released (useful for sustained throttle or setpoints).
- **Directional modes:** some configurations may emphasize horizontal vs. vertical control depending on the robot's requirements.

Typical joystick usage includes:

- Forward/backward movement (throttle)
- Left/right turning (steering)
- Camera pan/tilt (when mapped to a gimbal)

- Any continuous control channel required by the robot (e.g., setpoint control)

6.3 Switches (Discrete Channels)

Both left and right sides provide **toggle switches** for discrete control channels. Switches are intended for actions that are either *on/off* or select between a small number of states, such as:

- Enabling/disabling motors
- Selecting drive modes (e.g., slow/normal/boost)
- Turning auxiliary devices on/off (lights, pump, fan, etc.)

6.4 Knobs (Analog Channels)

Each side also includes a **rotary knob** for analog adjustment. Knobs are useful for fine-tuning parameters while driving:

- Steering sensitivity or trim
- Speed limit
- Camera zoom or focus (if supported)
- PID gains or other tuning parameters (advanced setups)

6.5 Center Buttons

The RC Screen includes **push buttons** near the center area (top and bottom per side). These are intended for quick actions that should be triggered immediately, for example:

- Horn / buzzer
- Reset / acknowledge
- Take picture / start recording (camera payload)
- Trigger an automated routine

6.6 Indicators and Telemetry

The interface provides several forms of feedback so the operator can monitor the system while controlling it:

- **Analog Indicator (top-left widget):** displays a numeric value with a title (e.g., RPM, temperature, signal strength).
- **Battery Status (top-right widget):** shows battery level with a title label.
- **LED Indicators:** a set of LEDs for quick status cues (e.g., armed state, warnings, link status).
- **Panel Indicators:** per-side panels showing a value, an on/off state, a color, and a title (useful for named telemetry channels).
- **Center Plot:** a live plot showing one or more telemetry series over time.

These indicators help confirm command reception and allow the operator to detect issues such as low battery, loss of link, or sensor warnings early.

6.7 Summary

In general:

- Use **joysticks** for continuous motion control.
- Use **switches and buttons** for discrete actions and mode changes.
- Use **knobs** for fine-grained tuning.
- Use **telemetry widgets and plots** to monitor system state while operating.

Chapter 7

Customizing Your Project

7.1 Adding Sensors

The firmware supports sensors connected to the ESP32.

Typical sensors include:

- Distance sensors
- Temperature sensors
- Analog sensors

Sensor data can be sent back to the Android application using telemetry.

7.2 Using I2C Devices

Additional devices can be connected using the I2C interface.

Examples include:

- MCP23017 GPIO expanders
- Environmental sensors
- Displays

7.3 Expanding Outputs

The system allows users to control multiple outputs.

Examples include:

- LEDs
- Relays
- Motor drivers
- Buzzers

Chapter 8

Example Projects

This chapter presents practical, real-life example projects that combine the two parts of the system:

- **EmitterApp (Android)**: provides the operator interface (dual joysticks, switches, knobs, event buttons) plus live telemetry widgets (panels, LED indicators, analog/battery indicators, and plots).
- **ESP32_BT_Controller (ESP32, C/C++)**: runs on the robot/controller hardware, receives RC commands over Bluetooth (SPP) and sends configuration + telemetry back to the app.

The goal of each example is to show (1) how the controls map to real hardware and (2) what telemetry is useful during operation.

8.1 RC Car (Differential Drive)

A common real-world build is a small **two-wheel differential drive** RC car/rover (two DC motors + motor driver). This is the simplest project that benefits immediately from the dual-stick interface.

Typical hardware

- ESP32 development board
- Dual H-bridge motor driver (e.g., TB6612FNG, DRV8833, L298N)
- 2 DC motors + chassis + battery
- Optional: battery voltage sensor (ADC divider)

Control mapping (practical)

- **Left stick Y**: throttle (forward/backward).
- **Left stick X**: steering (turn left/right) *or* "arcade mix" (throttle + steering combined into left/right motor speeds).
- **Switch 1**: arm/disarm motors (safety).
- **Knob**: max speed limiter (useful for indoor driving or training).

- **Event button**: horn/buzzer pulse (short audible feedback).

Telemetry that matters in real life

- **Battery indicator**: prevents over-discharging the pack.
- **Panels**: show left/right motor command or measured wheel speed (if encoders exist).
- **Plot**: battery voltage over time (sag under load shows when you are near depletion).

Real-life use case

Indoor navigation practice, basic robotics teaching labs, or a workshop demo car where operators can *feel* the difference between control modes (spring vs hold) while watching battery and load effects on the plot.

8.2 Camera Control Platform (Pan/Tilt Head)

A pan/tilt platform is a realistic project used for inspection, streaming, or robotics vision experiments. It is also a good example of mixing **continuous control** and **fine adjustment**.

Typical hardware

- ESP32 development board
- 2 servos (pan + tilt) or a small gimbal
- Camera (action camera, small FPV camera, phone mount, etc.)
- Optional: LED light for illumination

Control mapping (practical)

- **Right stick X**: camera pan.
- **Right stick Y**: camera tilt.
- **Knob**: servo speed/smoothing (slow cinematic motion vs fast tracking).
- **Switch**: toggle LED light (on/off).
- **Event button**: "center camera" action (send a short event to reset angles).

Telemetry that matters in real life

- **Analog indicator**: current pan angle or "target pan angle".
- **Panel**: tilt angle or a named mode (e.g., *TRACK* vs *LOCK*).
- **LED indicators**: show whether the platform is centered, tracking, or in a warning state.

Real-life use case

Home inspection (looking under furniture, ceilings), robotics vision tuning (point the camera while adjusting lighting), or a workshop live demo where one person drives and another controls the camera.

8.3 Sensor Rover (Telemetry-First Build)

A sensor rover emphasizes **measurement and feedback** more than speed. The ESP32 reads sensors and streams telemetry back to the Android device where the operator sees panels and time-series plots.

Typical hardware

- ESP32 development board
- Distance sensor (ultrasonic or ToF)
- Environmental sensor (temperature/humidity/pressure) over I2C
- Optional: MCP23017 I2C GPIO expander (if you need more IO)
- Rover chassis (2WD or 4WD)

Control mapping (practical)

- **Left stick**: slow drive (precision).
- **Switches**: toggle sensor modes (e.g., fast/slow sampling, logging on/off).
- **Knob**: sampling rate or alert threshold (e.g., distance warning limit).

Telemetry that matters in real life

- **Left panel**: distance (cm) to obstacle.
- **Right panel**: temperature (°C) or humidity (%).
- **Plots**: distance and/or voltage trends (useful for diagnosing noise, sensor dropouts, or power issues).
- **Battery indicator**: critical for longer experiments.

Real-life use case

Environmental mapping (walk the rover around a room to see temperature gradients), obstacle-distance experiments for autonomous driving prototypes, or demonstration of telemetry-driven operation where the plot reveals what is happening even when the robot is out of sight.

8.4 Custom Robotics Platform (Modular, Expandable)

A more advanced real-world project is a **modular robot** that mixes locomotion, actuators, and multiple accessory devices. This is where using both repositories together becomes especially

valuable: the Android side offers a rich operator UI, and the ESP32 side can be adapted (pin mapping, modes, I2C expansion, telemetry sources).

Typical hardware

- Drive base (tracked or wheeled)
- Actuator (servo arm, linear actuator, gripper)
- Lights + buzzer for feedback
- Multiple sensors (battery monitoring, encoders, distance, IMU)
- Optional: MCP23017 for extra switches/outputs

Example control mapping (practical)

- **Left stick**: drive.
- **Right stick**: arm control (rotate + lift) *or* camera control.
- **Switches**: modes (arm enable, precision mode, lights).
- **Knobs**: tuning (speed limit, grip force, servo trim).
- **Event buttons**: discrete actions (open/close gripper, reset orientation, emergency stop).

Telemetry that matters in real life

- **Battery widget**: operator safety (stop before brownouts).
- **Panels**: named signals such as *ARM CURR*, *IMU YAW*, *GRIP STATE*.
- **LEDs**: quick status (armed, warning, fault, link ok).
- **Plot**: one or more signals over time (e.g., current draw vs motor command to detect stalls).

Real-life use case

Maker projects and robotics clubs: one base platform can be reused for different events (sumo bot, inspection bot, outreach demo) by changing firmware configuration (pins/features) and reusing the same Android control screen.

8.5 Project Planning Checklist (Recommended)

Before building, decide:

- **Controls**: what each joystick axis, switch, knob, and button should do.
- **Safety**: an arm/disarm switch and an emergency-stop event is strongly recommended.
- **Telemetry**: pick 2–4 key values that make operation safer (battery, distance, motor current, temperature).
- **Expansion**: whether you need I2C devices (sensors, MCP23017) to scale IO.

Chapter 9

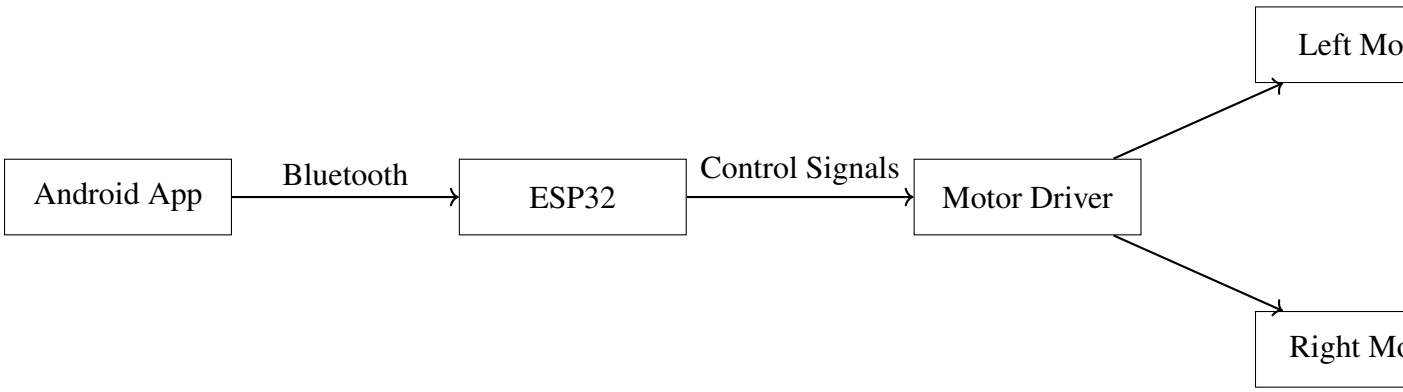
Wiring Diagrams

This chapter shows basic wiring examples for common projects using the ESP32 RC Controller firmware.

These diagrams are simplified so beginners can quickly understand how to connect the hardware.

9.1 Basic Robot Wiring

The most common project is a simple differential drive robot using two motors.



9.2 Servo Control Example

Servos can be connected directly to PWM pins on the ESP32.

- Servo signal → ESP32 GPIO pin
- Servo power → 5V supply
- Servo ground → common ground

Example uses:

- Steering control
- Camera pan control
- Robotic arm joints

9.3 LED and Buzzer Outputs

Digital outputs can control LEDs or buzzers.

Example connections:

- GPIO pin → LED (with resistor)
- GPIO pin → buzzer module

These outputs can be controlled using buttons or switches in the Android application.

9.4 Sensor Connections

Sensors can be connected using:

- Analog inputs
- Digital inputs
- I2C interface

The firmware allows sensor data to be transmitted to the Android application using telemetry.

Chapter 10

Troubleshooting

If your project does not work as expected, this chapter helps you identify common problems on **both sides of the system**:

- **Android (EmitterApp):** UI controls, Bluetooth pairing/connection, and whether commands/telemetry are being processed.
- **ESP32 (ESP32_BT_Controller):** firmware upload, pin wiring, power integrity, Bluetooth SPP link, and telemetry generation.

10.1 Before You Start: Quick Checklist

- Confirm the ESP32 is powered and stable (no brownouts/resets).
- Confirm your Android device is paired to the correct Bluetooth name (default is `ESP32-BT`).
- Keep the phone within a few meters for initial testing.
- Ensure **all grounds are common** (ESP32, motor driver, sensors, external 5V/servo supply).
- If you are using boot-sensitive pins (e.g., GPIO0), disconnect external circuits temporarily during flashing.

10.2 ESP32 Firmware Does Not Upload

If firmware does not upload correctly:

- Verify the correct board/target is selected in your IDE/tooling.
- Confirm the correct serial port is selected.
- Try a different USB cable (many "charge-only" cables cause upload failures).
- Press and hold **BOOT** during upload if your board requires manual bootloader entry.
- Disconnect external wiring from **boot-strap pins** during flashing (common offenders include GPIO0 and GPIO2).

Boot-strap pin notes (important)

The firmware pin configuration uses GPIO0 as a digital indicator input (`PIN_IND4 = GPIO0`) in `FULL_RC_MODE`. GPIO0 is a boot pin: external pull-ups/pull-downs can prevent boot or flashing. If flashing fails, unplug anything connected to GPIO0, flash again, then reconnect.

10.3 ESP32 Powers On but Resets Randomly

Random resets are usually power-related:

- Motors/servos can pull voltage down suddenly. Use a dedicated regulator for the ESP32 and a separate supply for motors/servos.
- Ensure the ESP32 supply can provide peak current (Wi-Fi/Bluetooth bursts can cause dips).
- Add bulk capacitance near motor drivers/servo rails (e.g., 470–1000 μF) and a smaller ceramic capacitor near the ESP32.
- Check for overheating regulators on dev boards when powering servos from the same 5V rail (not recommended).

10.4 Bluetooth Pairing and Connection Problems (Android ↔ ESP32)

Symptom: Device not visible in scan

- Confirm the ESP32 is powered and running (LED/activity indicators if available).
- Ensure the phone is close to the ESP32.
- Power-cycle the ESP32.

Symptom: Pairing asks for PIN or fails

The default PIN is commonly `1234`.

- Remove ("forget") the device in Android Bluetooth settings, then pair again.
- Try pairing in Android system settings first, then connect from the app.

Symptom: App says "Connected" but controls do nothing

- Verify you connected to the correct device (if multiple `ESP32-BT` devices exist nearby).
- Confirm the ESP32 is running the expected firmware build (not an old sketch).
- On the ESP32 side, use serial logging (if enabled) to confirm command packets are arriving.

Symptom: Connects, then disconnects repeatedly

- This is often a **power stability** problem on the ESP32 (see Section 10.3).

- Reduce motor/servo load during initial testing (disconnect motors, test with LEDs first).
- Keep the phone close, and avoid metal enclosures that shield Bluetooth.

10.5 Controls Feel Wrong in the App (Android Side)

Joystick returns to center vs stays in place

EmitterApp supports different joystick modes (for example spring vs hold). If the robot behaves as if throttle is "not staying" or steering is "stuck":

- Check the selected stick modes in the app settings.
- For throttle, many users prefer a **hold** behavior; for steering, many prefer **spring**.

Buttons/switches appear but do not affect hardware

- Confirm that the ESP32 firmware maps those channels to real pins/outputs in your chosen project mode.
- If you use `FULL_RC_MODE_MCP`, verify MCP23017 wiring and I2C address configuration (default address `0x20`).

10.6 Motors Do Not Move

Electrical checks

- Check motor driver power supply and polarity.
- Confirm the driver **logic ground** is connected to ESP32 ground.
- Confirm motor outputs are connected to the motor terminals (swap leads if direction is reversed).

Signal checks

- Confirm PWM pins match your wiring. In the default pin map, PWM channels include: GPIO18/19/23/25/26/27.
- Confirm direction pins are connected correctly: `PIN_MD_LEFT = GPIO12` and `PIN_MD_RIGHT = GPIO15` (both should be kept low at boot).
- If your driver needs two direction pins per motor (IN1/IN2), you must adapt the wiring/-firmware accordingly.

Practical debugging tip

Test motor outputs with a simple "LED + resistor" on the PWM pin first (you should see brightness change with stick motion), then connect the motor driver.

10.7 Servo Does Not Respond or Jitters

- Ensure the servo has a proper 5V supply (do **not** power medium/large servos from the ESP32 board).
- Confirm servo ground is tied to ESP32 ground.
- Verify the control signal is wired to a configured PWM output pin (e.g., GPIO23 or GPIO25).
- If the servo jitters, add a better power supply and reduce electrical noise (shorter wires, capacitors, separate power rails).

10.8 Telemetry Data Not Appearing in the App

Symptom: Connected, but no telemetry widgets update

- Wait a few seconds after connecting: the ESP32 sends configuration to the app on first connect.
- Verify the sensor is wired to the correct input type:
  - Analog telemetry: use ADC1 pins (e.g., GPIO34/35/36/39 in `FULL_RC_MODE`).
  - Digital indicators: GPIO4/5/2/0 (note GPIO0 boot caution).
  - I2C sensors: SDA GPIO21, SCL GPIO22.
- If you are using `FULL_RC_MODE_MCP`, confirm the MCP23017 is detected and wired correctly.

Symptom: Telemetry is present but values look wrong

- Check sensor scaling (voltage dividers, ADC range, units).
- Reduce noise: shorten analog wires, add filtering (RC filter), and avoid running analog lines alongside motor wires.

10.9 I2C Devices Not Detected (MCP23017 or Sensors)

- Verify SDA is on GPIO21 and SCL on GPIO22 (per `pins.h`).
- Confirm the device uses 3.3V logic (or use level shifting).
- Check pull-up resistors on SDA/SCL (many breakout boards include them; too many pull-ups in parallel can also cause issues).
- Confirm the I2C address (MCP23017 default in the repo is `0x20`).

10.10 General Debugging Tips (Recommended Workflow)

1. **Start with the link:** verify Bluetooth pairing and a stable connection first.
2. **Test outputs without motors:** LEDs on PWM/switch pins confirm the control path is working.

3. **Add one subsystem at a time:** motors, then servos, then sensors, then I2C expansion.
4. **Use logging:**
   - ESP32: serial debug output (when enabled in the firmware).
   - Android: Check the indicators to confirm connection state and incoming telemetry.
5. **Treat power as a first-class component:** most intermittent failures come from under-voltage or noisy supplies.

Chapter 11

Advanced Features

The **ESP32_BT_Controller** firmware is designed as a modular RC + telemetry platform. Beyond basic joystick control, it includes a lightweight communication protocol, configurable project modes, a structured telemetry system, and optional I2C expansion.

This chapter highlights the most useful advanced features and explains how they work in practice.

11.1 Protocol Overview: Commands, Events, and Telemetry

Communication is packet-based (see `packets.h`). The firmware uses:

- **RC State packets:** continuous control state (sticks, knobs, switches).
- **Event packets:** short one-shot commands (button presses).
- **Telemetry packets:** feedback from the ESP32 to the Android app (panels, indicators, plots, and configuration labels).

All packets use:

- Fixed headers (e.g., `0xCC` followed by a packet type byte)
- Packed layouts for predictable parsing
- A simple additive checksum (lightweight and fast for embedded systems)

11.2 Telemetry System (Panels, Indicators, Plots)

Telemetry is one of the most powerful parts of this firmware. It allows the ESP32 to send real-time values back to the Android app so the operator can *see what the robot is doing* while driving it.

11.2.1 Telemetry categories

The firmware generates multiple telemetry streams (implemented in `telemetry.cpp`):

- **Panels** (left/right numeric values): compact readings such as distance, speed, temperature, or any sensor-derived number.
- **Indicators**: a small set of "status widgets" including:

  - an analog value (0–100)
  - a battery level (0–100)
  - a digital mask (bitfield) for multiple on/off indicators

- **Plots**: fast-updating series for time graphs (default labels include `Volts`, `Amps`, and `RPMs`).

11.2.2 Telemetry update rates (practical meaning)

Telemetry is sent periodically (timing constants in `telemetry.cpp`):

- **Plots:** every 50 ms ($\approx 20\,Hz$) — smooth graphs for fast-changing signals.
- **Indicators:** every 500 ms ($\approx 2\,Hz$) — stable UI values for battery/status.

11.2.3 Automatic configuration handshake (labels)

When the phone connects, the ESP32 sends a **configuration telemetry packet once per connection** (`sendConfigTelemetry()`). This packet provides the app with human-readable labels for:

- plot series names (e.g., `Volts`, `Amps`, `RPMs`)
- panel names (e.g., `Left`, `Right`)
- indicator names (e.g., `Throttle`, `Battery`)

This is a major usability feature: the Android app can populate labels automatically without hardcoding them for each robot build.

11.2.4 Panel resend window (robustness feature)

Panel values use a small resend mechanism ("retry window") so that when a panel reading changes, it is retransmitted repeatedly for a short period (see `resendWindow` and `resendInterval` in `telemetry.cpp`). This improves the chance that the phone receives the updated reading even if a packet is missed.

11.3 Telemetry Sources and Debug Injection

Telemetry is intentionally split into two layers:

- **`telemetry.cpp`** is *write-only*: it builds packets and sends them.
- **`telemetry_source.cpp`** decides *where data comes from*: hardware reads or debug values.

In normal operation, panel/indicator/plot values are read from hardware through `input_hw.h` functions (e.g., numeric inputs and ADC readings). In debug modes, values can be injected over serial by typing simple comma-separated values. This makes it possible to verify Android UI behavior and packet decoding *without* real sensors connected.

Why this matters: You can develop UI + telemetry displays first, then integrate hardware later, reducing iteration time.

11.4 Project Modes: ESP32-Only vs MCP23017 Expansion

The firmware can be built in different project modes (configured in `user_config.h`):

- `MODE_FULL_RC`: full RC control using ESP32 GPIO directly.
- `MODE_FULL_RC_MCP`: full RC control using an MCP23017 over I2C for expanded digital I/O.

Selecting the correct mode is an "advanced feature" because it allows a single codebase to scale from small robots to larger builds with many switches and indicators.

11.5 Using the MCP23017 GPIO Expander (I2C)

The MCP23017 adds 16 digital I/O pins over I2C and is supported by the firmware (`mcp_io.cpp/.h`).

11.5.1 Key details (from the implementation)

- **I2C address:** `0x20`
- **Port A (GPIOA):** configured as **INPUT** (used for reading digital indicators)
- **Port B (GPIOB):** configured as **OUTPUT** (used for switch and event outputs)
- Outputs are tracked in software (`portBState`) so individual pins can be toggled reliably.

11.5.2 Practical applications

- A control box with many switches (arm, lights, modes, accessories)
- A robot with many status inputs (limit switches, fault lines, bumper sensors)
- Complex LED indicator arrays (without consuming ESP32 GPIO)

11.6 I2C Sensor Support (Examples Included)

The repository includes example I2C sensor reads in `i2c_sensors.cpp`:

- A temperature sensor at address `0x48`
- An IMU example at address `0x68` (accelerometer reads, e.g., MPU6050-style registers)

These examples act as templates: you can add initialization steps and additional registers to support your exact sensor module.

11.7 Analog Input Design for Bluetooth Compatibility

A subtle but important advanced design choice is the use of **ADC1-only** inputs for analog reads in the default pin configuration. On ESP32, ADC2 is commonly problematic when Wi-Fi/Bluetooth is active, so the repo pin map keeps analog telemetry on ADC1-capable pins (see `pins.h`).

Practical implication: If you add analog sensors, prefer the configured ADC1 pins (or other ADC1 channels) to avoid intermittent or invalid readings while Bluetooth is connected.

11.8 Feature Flags (Build-Time Modular Firmware)

Build-time feature flags are defined in `feature_config.h`. They control whether subsystems are compiled/enabled (PWM outputs, switch outputs, event outputs, telemetry, I2C sensor features, etc.).

Note: In the current repository snapshot, `USE_TELEMETRY` and `USE_I2C_SENSOR` are defined as `0` in both project modes, while telemetry implementation code exists in the repo. This suggests telemetry can be enabled/disabled by configuration depending on project needs (e.g., to reduce bandwidth or CPU use).

11.9 Putting It All Together: Example "Advanced Build"

A realistic advanced project combines these features:

- **RC control:** dual sticks + knobs for driving and camera/arm control
- **MCP23017:** many switches and indicators using I2C expansion
- **Telemetry:** battery + current plots, plus panels for temperature and distance

This creates a robust operator experience: control remains responsive, while telemetry provides situational awareness and early warning of battery sag, motor stalls, or sensor faults.

Chapter 12

Practical Control and Telemetry Examples

12.1 Introduction

This chapter provides practical examples that demonstrate how the Android application and the **MICSBOL/ESP32_BT_Controller** firmware interact in a real robot or embedded system.

The Android side continuously transmits a **control state** (sticks, knobs, and switches), and also transmits **one-shot events** when the user presses buttons.

The ESP32 side:

- decodes incoming packets over Bluetooth Serial (SPP),
- applies the received control values to real hardware outputs (PWM, direction pins, switches),
- and optionally sends telemetry back to the Android app (panels, indicators, plots).

12.2 Control Data from the Android Application

12.2.1 RC state packet (`rcStatePacket`)

The firmware stores the most recent control state in the structure defined in `packets.h`:

```
rcStatePacket
```

It contains the following fields:

- `leftStickX`, `leftStickY`
- `rightStickX`, `rightStickY`
- `leftKnob`, `rightKnob`
- `switches` (bit mask)

In the repository code, the stick and knob channels are treated as 12-bit values and are mapped in the range **0..4095** (see `control.cpp`, `hal_output.cpp`).

12.2.2 Event packet (`rcEventPacket`)

Buttons are transmitted as small event packets, and the event ID is passed to:

```
controlHandleEvent(byte eventId)
```

Events are useful for actions that should be *momentary* (e.g., trigger, reset, horn) rather than a continuous analog value.

12.3 How the ESP32 Receives Packets

Incoming Bluetooth bytes are processed in:

```
receiver.cpp  ->  handleBluetooth()
```

The receiver maintains a rolling buffer and searches for packet headers. In the current firmware implementation:

- RC State header: `0xAA 0x55`
- Event header: `0xBB 0x66`

Once an event is decoded, `controlHandleEvent()` is called immediately.

12.4 Where Control Values are Applied

The main control loop is implemented in:

```
control.cpp  ->  controlUpdate()
```

This function reads values from `rcStatePacket` and sends them to the hardware abstraction layer (`hal_output.cpp`):

- steering servo PWM
- left/right motor commands
- camera pan PWM
- LED brightness and buzzer intensity (PWM)
- switch outputs (bit mask decoded into per-channel outputs)

12.5 Example 1: Steering Control Using the Left Joystick

The firmware maps the left joystick horizontal axis to a servo-like PWM value:

```
rcStatePacket.data.leftStickX
```

In `control.cpp`, steering is generated by:

```
static uint32_t mapServo(uint16_t v) {
    return map(v, 0, 4095, 205, 410);
}

uint32_t steerPWM = mapServo(rcStatePacket.data.leftStickX);
halSetSteering(steerPWM);
```

Practical interpretation

- The Android joystick produces a value from 0 to 4095.
- The ESP32 maps it into a smaller PWM range (205..410) suitable for its PWM configuration.

This is commonly used for:

- RC car steering
- robot front-wheel steering
- a servo-actuated rudder

12.6 Example 2: Differential Drive Motor Speed Control

The firmware uses the vertical joystick axes as left/right motor channels:

```
rcStatePacket.data.leftStickY
rcStatePacket.data.rightStickY
```

In `control.cpp`:

```
uint32_t motorL = rcStatePacket.data.leftStickY;
uint32_t motorR = rcStatePacket.data.rightStickY;

halSetMotorLeft(motorL);
halSetMotorRight(motorR);
```

12.6.1 Direction + speed mapping (implemented behavior)

In `hal_output.cpp`, each motor value is interpreted with a dead-band:

- **0..2000:** direction pin LOW (reverse) and PWM mapped to 0..4095
- **2001..2094:** dead-band, motor target set to 0
- **2095..4095:** direction pin HIGH (forward) and PWM mapped to 0..4095

This design reduces jitter when the joystick is near center.

12.6.2 Smooth ramping (advanced behavior)

Instead of writing PWM instantly, the motor PWM ramps toward a target value. A timer interrupt updates the motor outputs every 20 ms, moving speed by a fixed step (see `updateMotors()` in `hal_output.cpp`). This improves drivability and reduces current spikes.

12.7 Example 3: Camera Pan Control

The right joystick horizontal axis is used for camera pan:

```
uint32_t panPWM = mapServo(rcStatePacket.data.rightStickX);
halSetCameraPan(panPWM);
```

Typical real-life uses:

- inspection rover camera
- FPV camera positioning
- turret-like sensor mount

12.8 Example 4: Using Knobs to Control LED and Buzzer PWM

The Android app provides two knobs, transmitted as 0..4095 values:

```
rcStatePacket.data.leftKnob
rcStatePacket.data.rightKnob
```

In `control.cpp`, these map directly to PWM outputs via the HAL:

```
halSetLed(rcStatePacket.data.leftKnob);
halSetBuzzer(rcStatePacket.data.rightKnob);
```

Practical uses

- LED brightness (headlights, underglow, work light)
- buzzer loudness / intensity (alerts, confirmation beeps)
- repurpose knobs for motor limit scaling or servo trim (custom firmware modification)

12.9 Example 5: Controlling Digital Outputs with Switches

Switches are transmitted as a single byte bit mask:

```
byte sw = rcStatePacket.data.switches;
```

In `control.cpp`, the firmware iterates through switch indices:

```
for (int i = 0; i < 6; i++) {
    halSetSwitch(i, (sw >> i) & 1);
}
```

Important mode difference

The number of switches supported depends on project mode (see `hal_output.cpp`):

- **FULL_RC_MODE:** only indices 0..3 are written to GPIO pins (4 switch outputs).
- **FULL_RC_MODE_MCP:** indices 0..5 are written using MCP23017 outputs (6 switch outputs).

Real-world uses:

- enable/arm switch (safety)
- headlights / beacon / underglow
- relay outputs (pump, solenoid, accessory power)
- mode selection (manual vs assisted)

12.10 Example 6: Handling Button Events (Pulse Outputs)

Events are handled by:

```
controlHandleEvent(byte eventId)
```

In the repository implementation, events call `halPulseSwitch(index)`. The index mapping differs by project mode. For example, in `FULL_RC_MODE`:

```
case 0x01: halPulseSwitch(0); break;
case 0x02: halPulseSwitch(1); break;
case 0x03: halPulseSwitch(0); break;
case 0x04: halPulseSwitch(1); break;
```

12.10.1 Non-blocking pulses

`halPulseSwitch()` uses `pulseStart()` which generates a **non-blocking pulse** (default 50 ms) and requires `pulseUpdate()` to be called in the main loop.

Typical event actions:

- horn / short sound effect
- "take picture" trigger for a camera
- reset a subsystem (IMU re-zero, controller reset line)
- momentary relay (door latch, actuator kick)

12.11 Telemetry Data Sent to the Android Application

Telemetry is generated by:

```
telemetry.cpp
telemetry_source.cpp
```

The design is intentionally split:

- **`telemetry.cpp`** builds and sends packets (write-only).
- **`telemetry_source.cpp`** provides values (hardware reads or debug injected values).

12.11.1 Telemetry types and rates (implemented)

- **Indicators:** every 500 ms ($\approx 2\,Hz$).
- **Plots:** every 50 ms ($\approx 20\,Hz$).
- **Panels:** retransmitted briefly when values change (resend window for robustness).

12.11.2 Config-on-connect

On a new connection, the ESP32 sends a **configuration packet** that includes plot/panel/indicator names (e.g., `Volts`, `Amps`, `RPMs`, `Left`, `Right`, `Throttle`, `Battery`). This allows the Android app to label UI elements automatically.

12.12 Example 7: Battery Telemetry as an Indicator

The firmware's indicator packet includes:

- `analogValue` (0–100)
- `batteryLevel` (0–100)
- `digitalMask` (bitfield)

In normal mode, battery is read from an ADC pin and mapped to a percentage (see `hwReadIndicatorBattery(` in `input_hw.cpp`):

```
return map(analogRead(PIN_ANALOG_IND2), 0, 4095, 0, 100);
```

Practical wiring note: battery voltage must be scaled with a voltage divider so the ESP32 ADC never exceeds 3.3V.

12.13 Example 8: Telemetry Panels from Analog Inputs

Panel telemetry uses two 16-bit values (`leftPanelValue`, `rightPanelValue`). The default hardware implementation reads analog values and scales them from 0..4095 to 0..65535:

```
uint16_t hwReadNumeric1() {
    uint32_t v = analogRead(PIN_NUMERIC1);
    return (uint16_t)((v * 65535UL) / 4095);
}
```

This is suitable for:

- distance sensors with analog output
- potentiometer feedback
- scaled sensor signals (after conditioning)

12.14 Example 9: Plot Signals (0..255 Fast Graphs)

Plots transmit three 8-bit values (0..255) at $\approx 20\,Hz$. The default hardware implementation reads ADC values and maps them to 0..255:

```
uint8_t hwReadPlot1() { return map(analogRead(PIN_NUMERIC1), 0, 4095, 0, 255); }
uint8_t hwReadPlot2() { return map(analogRead(PIN_NUMERIC2), 0, 4095, 0, 255); }
uint8_t hwReadPlot3() { return map(analogRead(PIN_ANALOG_IND2), 0, 4095, 0, 255)
```

Practical plot ideas:

- battery voltage trend (sag under acceleration)
- current sensor trend (detect stalls)
- RPM/encoder-derived speed trend

12.15 Example 10: Digital Mask Indicators

The indicator telemetry includes a digital mask byte that can represent up to 8 on/off signals. In `input_hw.cpp`, a mask is built from indicator GPIO reads (or from MCP23017 inputs in MCP mode).

This is useful for compactly showing:

- limit switches
- fault flags
- bumper sensors
- "armed" / "ready" signals

12.16 Summary

The system supports bidirectional communication:

- **Android → ESP32:** continuous RC state + discrete events.
- **ESP32 → Android:** panels, indicators, and plots with automatic label configuration.

By combining control inputs (sticks, knobs, switches, events) with live telemetry feedback, you can build robots and embedded systems that are both **easy to operate** and **easy to debug in real time**.

Chapter 13

Control Layer and Hardware Abstraction

13.1 Purpose of This Chapter

This chapter documents how the **MICSBOL/ESP32_BT_Controller** firmware separates:

- **control logic** (what the robot should do when the user moves a stick), and
- **hardware driving** (how the ESP32 writes PWM/GPIO/I^2C signals).

The repository implements this separation primarily through:

- `control.cpp` / `control.h` — the control layer
- `hal_output.cpp` — the hardware abstraction layer (HAL)
- `pins.h` — the hardware mapping used by the HAL
- `pulse.cpp` / `pulse.h` — non-blocking pulse outputs used by event actions

13.2 Layered Architecture (High-Level View)

The firmware uses a layered flow from the Android app to physical hardware:

1. **Android app** transmits RC state packets and event packets over Bluetooth (SPP).
2. **Receiver layer** (`receiver.cpp`) decodes packets and updates:
   - `rcStatePacket` for continuous control values
   - `rcEventPacket` for one-shot button events
3. **Control layer** (`control.cpp`) interprets `rcStatePacket` and calls HAL functions.
4. **HAL** (`hal_output.cpp`) performs the actual GPIO/PWM/I^2C operations.

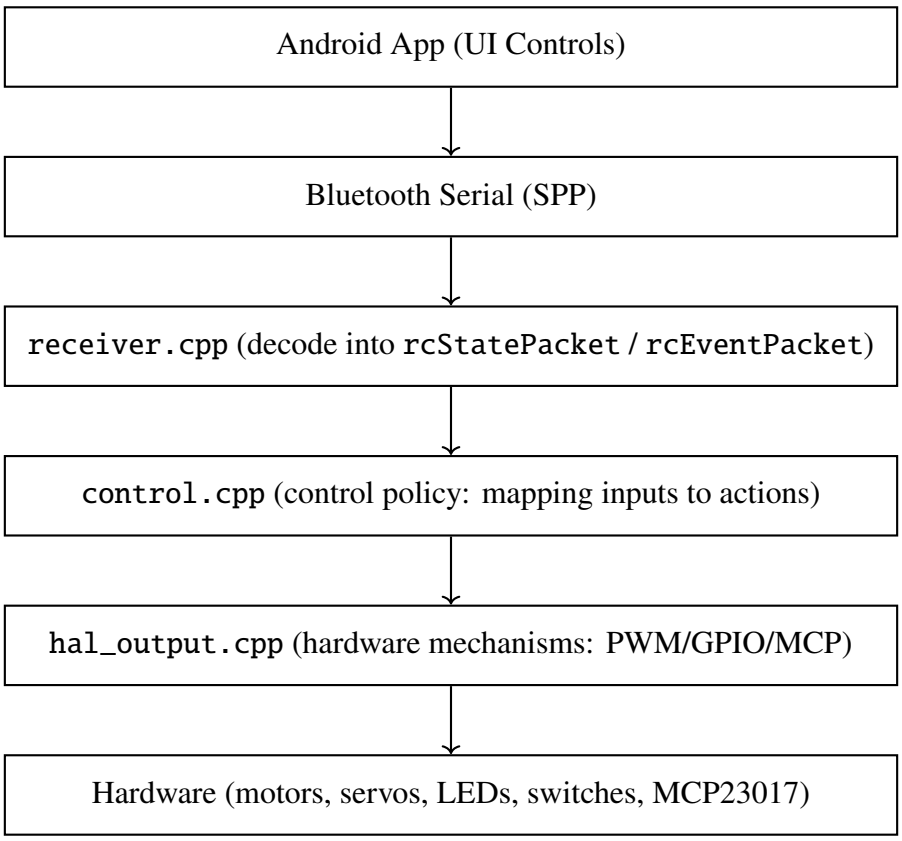


Figure 13.1: Control data flow from Android to ESP32 hardware.

13.3 Control Packets and State Structures

Control values are stored in `rcStatePacket` (defined in `packets.h`). The fields include:

- `leftStickX`, `leftStickY`
- `rightStickX`, `rightStickY`
- `leftKnob`, `rightKnob`
- `switches` (bit mask)

In the current firmware logic, stick and knob channels are treated as 12-bit values (nominal range 0 to 4095).

Button presses are delivered through `rcEventPacket` (also in `packets.h`) and are handled by `controlHandleEvent(eventId)`.

13.4 Control Layer (`control.cpp`)

13.4.1 Responsibilities

The control layer is responsible for:

- mapping sticks and knobs to actuator commands,
- selecting which HAL function to call for each logical output,
- decoding the switch bit mask into per-output states,

- handling discrete button events by triggering pulse outputs.

13.4.2 Main loop: `controlUpdate()`

The main control function is:

```
controlUpdate()
```

In the repository implementation, `controlUpdate()` reads values from `rcStatePacket` and calls:

- `halSetSteering(...)`
- `halSetMotorLeft(...)`
- `halSetMotorRight(...)`
- `halSetCameraPan(...)`
- `halSetLed(...)`
- `halSetBuzzer(...)`
- `halSetSwitch(index, state)`

A key helper is the servo mapping function in `control.cpp`:

```
static uint32_t mapServo(uint16_t v) {
    return map(v, 0, 4095, 205, 410);
}
```

13.4.3 Switch bit mask decoding

Switches are transmitted as a single byte `rcStatePacket.data.switches`. The control layer converts bits into outputs:

```
for (int i = 0; i < 6; i++) {
    halSetSwitch(i, (sw >> i) & 1);
}
```

13.4.4 Event handling: `controlHandleEvent(eventId)`

Discrete events (button presses) are handled by:

```
controlHandleEvent(byte eventId)
```

In the repository, events map to pulse outputs via `halPulseSwitch(index)`. The mapping depends on project mode (direct GPIO vs MCP expander mode).

13.5 Hardware Abstraction Layer (`hal_output.cpp`)

13.5.1 Responsibilities

The HAL implements the *mechanisms* to drive hardware:

- PWM output via ESP32 LEDC using `ledcWrite(...)`
- GPIO output via `digitalWrite(...)`

- MCP23017 expander output via `mcpWriteOutput(...)` (I^2C)
- motor direction pins and a dead-band mapping
- ramping motor speed using a periodic timer callback
- non-blocking pulse outputs using the pulse module

13.5.2 PWM outputs (examples)

Examples from `hal_output.cpp`:

```
void halSetSteering(uint32_t value) { ledcWrite(PIN_PWM_STICK_LX, value); }
void halSetLed(uint32_t value)      { ledcWrite(PIN_PWM_KNOB_L, value);  }
void halSetBuzzer(uint32_t value)   { ledcWrite(PIN_PWM_KNOB_R, value);  }
```

13.5.3 Motor control: direction + dead-band + ramping

Motor control in the HAL uses:

- direction pins `PIN_MD_LEFT` and `PIN_MD_RIGHT`
- PWM output pins `PIN_PWM_STICK_LY` (left) and `PIN_PWM_STICK_RY` (right)

The HAL interprets motor values with a dead-band around the center, and it ramps actual motor PWM toward a target value using a timer-driven update function.

13.5.4 Switch outputs: direct GPIO vs MCP23017

The HAL provides a uniform interface:

```
halSetSwitch(uint8_t index, bool state)
```

but the implementation depends on build mode:

- In direct GPIO mode, a small array of ESP32 pins is written using `digitalWrite(...)`.
- In MCP mode, the output is written using `mcpWriteOutput(...)` over I^2C.

13.5.5 Pulse outputs (momentary actions)

Pulse outputs are triggered by `halPulseSwitch(index)` and implemented using:

- `pulseStart(pin, durationMs)` to begin a pulse
- `pulseUpdate()` to end it after the requested duration

This design is non-blocking, so the control loop can continue running while pulses are active.

13.6 Pin Mapping (`pins.h`) and Portability

Hardware mappings are centralized in `pins.h`. When adapting the firmware to new hardware:

1. update `pins.h` to match your wiring,
2. adjust `hal_output.cpp` if your motor driver or output logic differs,
3. keep `control.cpp` stable as “policy” whenever possible.

13.7 Summary

The repository implements a clear separation between:

- **Policy:** `control.cpp` interprets `rcStatePacket` and decides what actions to take.
- **Mechanism:** `hal_output.cpp` drives PWM/GPIO/I^2C hardware to perform those actions.

This makes the firmware easier to port across different robots and wiring layouts, while keeping the codebase organized and maintainable.

Chapter 14

Wiring Diagram Explained (Repository Reference)

14.1 Quick Summary

Figure 14.1 is a practical wiring reference for the **MICSBOL/ESP32_BT_Controller** project. It shows:

- **Inputs (left side):**
  - I2C bus (SDA/SCL) for sensors or expansion
  - Analog inputs (NUM1/NUM2/AIN1/AIN2) used as telemetry sources
  - Digital indicator inputs (IND1–IND4) used to build a status mask
- **Outputs (right side):**
  - PWM outputs for servos, motors, LED brightness and buzzer
  - Switch outputs (SW1–SW4 in direct mode, SW1–SW6 in MCP mode)
  - Pulse/event outputs used for momentary actions (button events)

In code terms:

- `control.cpp` maps `rcStatePacket` fields to actions (steer, motors, pan, LED, buzzer, switches).
- `hal_output.cpp` performs the real hardware writes (PWM via `ledcWrite`, GPIO via `digitalWrite`, MCP outputs via `mcpWriteOutput`, pulses via `pulseStart`).
- `input_hw.cpp` reads ADC and indicator pins (used for telemetry sources).
- `pins.h` defines the GPIO mapping (and it changes depending on the project mode).

14.2 Diagram (Included)

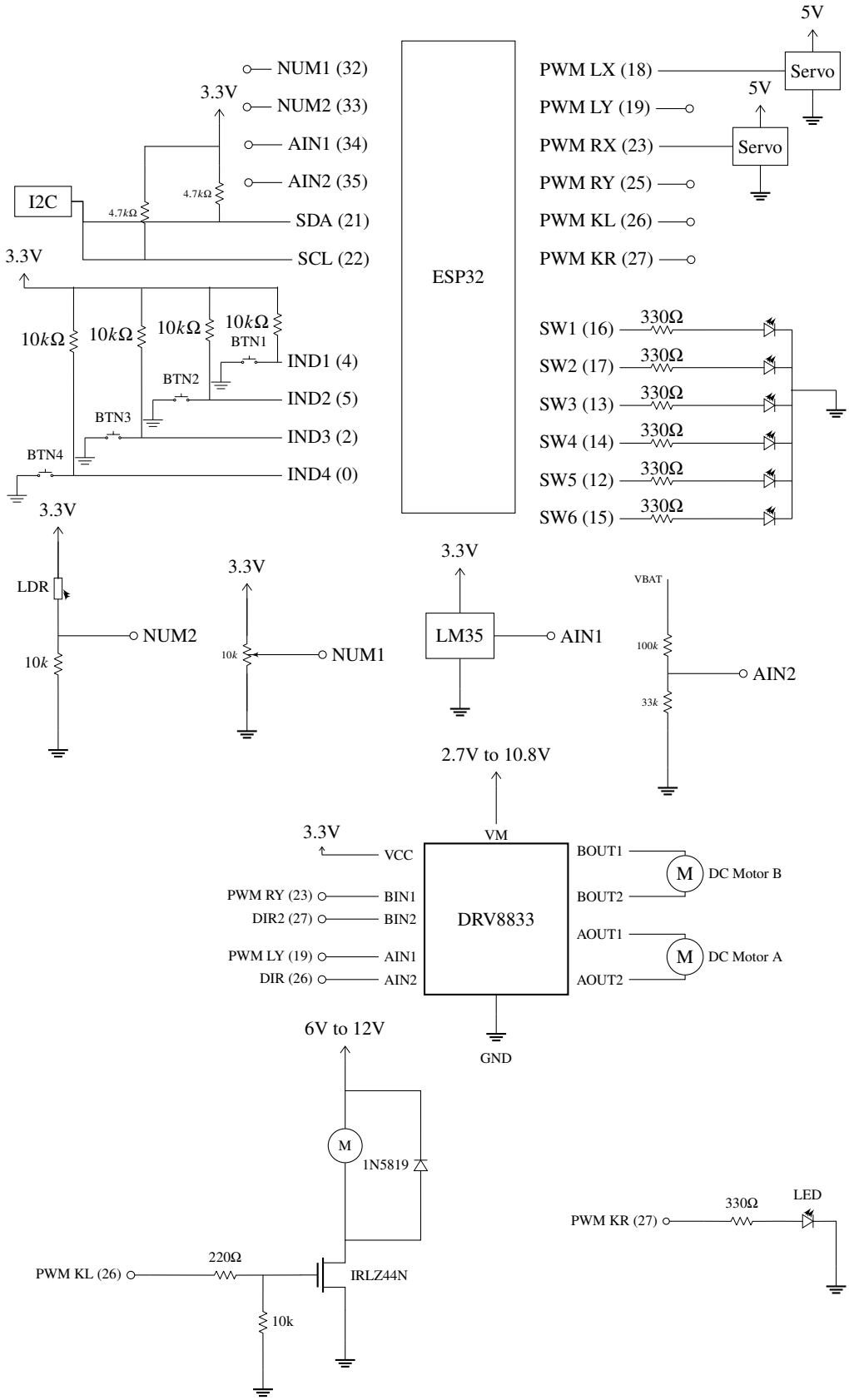


Figure 14.1: Suggested ESP32 pin assignment and peripheral connections for the `FULL_RC_MODE` configuration. The shown circuits are examples and can be modified according to the user's specific application requirements.

14.3 Straight Explanation of Each Group (What it does)

14.3.1 I2C (SDA 21, SCL 22)

Used to connect external I2C devices. In the repository, the same bus is also used by the `FULL_RC_MODE_MCP` configuration to talk to an MCP23017 expander (address `0x20` in `pins.h`).

14.3.2 Analog telemetry inputs (NUM1/NUM2/AIN1/AIN2)

These are ADC inputs read in `input_hw.cpp` using `analogRead(...)`. They are intended for telemetry sources such as battery voltage (through a divider), temperature sensors, light sensors, potentiometers, etc.

14.3.3 Digital indicator inputs (IND1–IND4)

These are digital inputs (GPIO) that represent on/off statuses (limit switch, fault line, bumper, etc.). In `input_hw.cpp` they are combined into a bit mask so multiple statuses can be sent to the Android UI.

14.3.4 PWM outputs (PWM LX/LY/RX/RY/KL/KR)

PWM outputs are driven by `hal_output.cpp` using `ledcWrite(...)` and are fed by values from `control.cpp`. Typical uses in the repository wiring concept:

- servos (steering, camera pan)
- motor speed PWM
- LED brightness
- buzzer intensity

14.3.5 Switch outputs (SW1–SW6)

These are digital outputs controlled by switch bits received from the Android app (`control.cpp` calls `halSetSwitch(index, state)`).

- Direct mode provides 4 physical outputs (`PIN_SW1`..`PIN_SW4`).
- MCP mode provides 6 outputs via MCP23017 (`GPB0_SW1`..`GPB5_SW6`).

14.3.6 Pulse/event outputs

Android button events are handled by `controlHandleEvent(eventId)` in `control.cpp`, which triggers momentary pulses via `halPulseSwitch(index)`. Pulses are implemented non-blocking in `pulse.cpp`. These outputs are ideal for triggers (horn, latch, reset) rather than continuous states.

14.4 Resume (One Paragraph)

The wiring diagram groups the ESP32 connections exactly the way the repository code is organized: `control.cpp` decides what each input means, `hal_output.cpp` drives PWM and GPIO outputs, `input_hw.cpp` reads analog and indicator inputs for telemetry, and `pins.h`

defines the GPIO mapping. By keeping these responsibilities separated, you can adapt the hardware (change pin numbers, add I2C devices, switch to MCP23017) without rewriting the overall control logic.